

---

# Setting up and running the DASH Repository mockup

How everything is wired, how to operate it, and how to move it

Charles Perkins Centre, Data Science Hub  
University of Sydney

June 2026

*This document is the operating manual for the DASH Repository mockup as it exists at the end of Phase 1's build phase. It assumes you can navigate a terminal and have basic familiarity with git, but it does not assume prior knowledge of Cloudflare Workers, MongoDB Atlas, or any of the specific services involved. The aim is that whoever picks this up next, you or someone else, can get from a fresh laptop to a working deployment without needing to ask.*

## Contents

|          |                                                              |          |
|----------|--------------------------------------------------------------|----------|
| <b>1</b> | <b>What this document is</b>                                 | <b>1</b> |
| <b>2</b> | <b>The system at a glance</b>                                | <b>2</b> |
| 2.1      | The six pieces . . . . .                                     | 2        |
| 2.2      | What happens on a single search . . . . .                    | 2        |
| <b>3</b> | <b>Initial setup: the accounts and credentials</b>           | <b>3</b> |
| 3.1      | Cloudflare account . . . . .                                 | 3        |
| 3.2      | MongoDB Atlas account and cluster . . . . .                  | 3        |
| 3.3      | Apply the Atlas indexes . . . . .                            | 4        |
| 3.4      | GitHub repository . . . . .                                  | 4        |
| 3.5      | Generate the ingest secret . . . . .                         | 4        |
| 3.6      | Summary of what you should now have . . . . .                | 5        |
| <b>4</b> | <b>Deploying the API Worker</b>                              | <b>5</b> |
| 4.1      | Install Wrangler . . . . .                                   | 5        |
| 4.2      | Authenticate Wrangler with your Cloudflare account . . . . . | 5        |
| 4.3      | Set the Worker secrets . . . . .                             | 5        |
| 4.4      | Deploy the Worker . . . . .                                  | 6        |
| 4.5      | Smoke test the deployed Worker . . . . .                     | 6        |
| 4.6      | Load initial project data . . . . .                          | 6        |
| <b>5</b> | <b>Deploying the frontend</b>                                | <b>6</b> |
| 5.1      | Wire the frontend to the API . . . . .                       | 7        |
| 5.2      | Build and deploy the frontend Worker . . . . .               | 7        |
| 5.3      | Test a query end to end . . . . .                            | 7        |
| 5.4      | Setting up the deploy via Cloudflare's dashboard . . . . .   | 7        |
| <b>6</b> | <b>CI/CD: GitHub Actions for the API and the catalog</b>     | <b>8</b> |

|           |                                                       |           |
|-----------|-------------------------------------------------------|-----------|
| 6.1       | The two workflows in plain language . . . . .         | 8         |
| 6.2       | Create a Cloudflare API token . . . . .               | 8         |
| 6.3       | Add the four secrets to GitHub . . . . .              | 8         |
| 6.4       | Verify the pipeline . . . . .                         | 8         |
| <b>7</b>  | <b>Day-to-day operations</b>                          | <b>9</b>  |
| 7.1       | Adding or updating a project . . . . .                | 9         |
| 7.2       | Editing the API code . . . . .                        | 9         |
| 7.3       | Editing the frontend . . . . .                        | 10        |
| 7.4       | Running the smoke test . . . . .                      | 10        |
| 7.5       | Viewing logs . . . . .                                | 10        |
| <b>8</b>  | <b>Migrating to DASH-owned infrastructure</b>         | <b>11</b> |
| 8.1       | What moves and what does not . . . . .                | 11        |
| 8.2       | The two Atlas options . . . . .                       | 11        |
| 8.3       | The full migration sequence . . . . .                 | 11        |
| 8.4       | Why this is straightforward . . . . .                 | 12        |
| <b>9</b>  | <b>Troubleshooting</b>                                | <b>12</b> |
| 9.1       | The frontend loads but no results come back . . . . . | 12        |
| 9.2       | The deploy workflow fails . . . . .                   | 12        |
| 9.3       | The ingest workflow fails . . . . .                   | 13        |
| 9.4       | Atlas connection refused or timeout . . . . .         | 13        |
| <b>10</b> | <b>Reference</b>                                      | <b>13</b> |
| 10.1      | File and directory layout . . . . .                   | 13        |
| 10.2      | HTTP endpoints exposed by the API Worker . . . . .    | 14        |
| 10.3      | Secrets and where they are set . . . . .              | 14        |
| 10.4      | What goes into the embedding source text . . . . .    | 15        |
| 10.5      | Schema enums . . . . .                                | 15        |

## 1 What this document is

This document describes a working mockup of the DASH Repository: a search system that lets researchers ask natural-language questions about past DASH consultations and get back the most relevant past projects. The mockup is live at the time of writing, with a frontend, an API server, a database, and a CI/CD pipeline all wired together.

You should read this document if you need to:

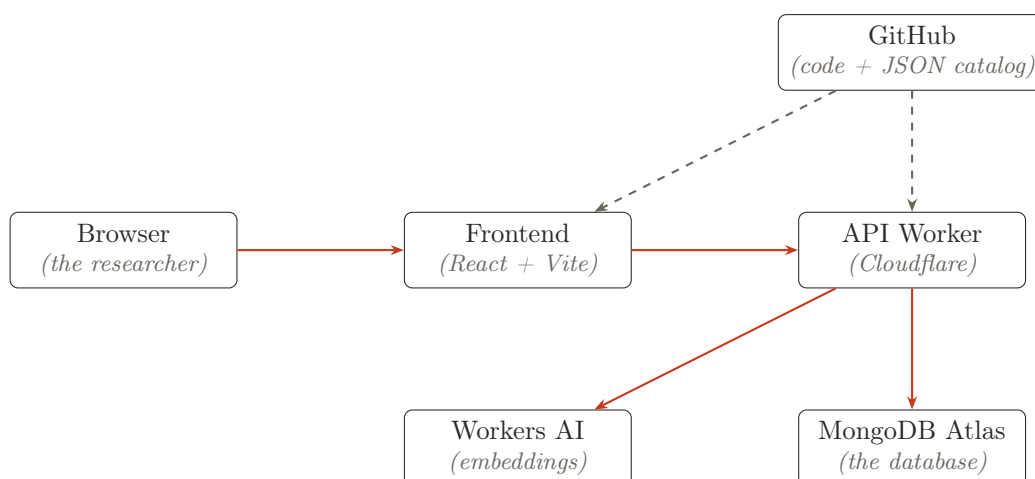
- Understand how the system is put together (Section 2).
- Set the whole thing up from scratch on a new set of accounts (Sections 3 to 6).
- Add a new project to the catalogue, or edit the code (Section 7).
- Move the mockup off your personal Cloudflare or Atlas accounts onto DASH-owned infrastructure (Section 8).
- Figure out why something has stopped working (Section 9).
- Look up where a specific file lives or which environment variable controls what (Section 10).

The document is written so it can be read top to bottom on a first pass, then used as a reference afterwards. If you only have time to read one section, read Section 2.

## 2 The system at a glance

The mockup has six pieces working together. Five of them are services run by other people that we pay for or use for free; the sixth is a stateless server we operate ourselves.

### 2.1 The six pieces



*Solid arrows show the live request path during a search. Dashed arrows show the deploy path: GitHub Actions ships code changes to the Worker and the frontend.*

The six pieces and what each one does:

1. **Browser.** What the researcher uses. They open a URL, they type a question, they see results. We do not operate this; the user's own laptop runs it.
2. **Frontend.** A small static web app written in React with Vite as the build tool. It is the page the researcher actually sees. It calls the API for results and renders them. It is hosted on Cloudflare and served from `https://dash-frontend.<subdomain>.workers.dev`.
3. **API Worker.** A small server written in JavaScript that runs on Cloudflare's edge network. It exposes a few HTTP endpoints. It takes a query from the frontend, asks Workers AI to turn it into a vector, asks MongoDB Atlas to find the closest matching projects, and returns the results. It is hosted on Cloudflare at `https://dash-api.<subdomain>.workers.dev`.
4. **Workers AI.** Cloudflare's embedding service. We send it a piece of text, it sends back a list of 1024 numbers (a vector) that represents the meaning of the text. We use the model `bge-large-en-v1.5`. We do not operate this.
5. **MongoDB Atlas.** The database. It holds one document per project, including the metadata, the executive summary, and the embedding vector. It also performs the vector similarity search at query time. We do not operate the database itself, but we own the data and the schema.
6. **GitHub.** Where the code lives and where new project metadata is added. Pushing to GitHub triggers GitHub Actions, which redeploys the API Worker (if the API code changed) or pushes new project metadata into MongoDB (if a project JSON changed).

## 2.2 What happens on a single search

Walking through a query from end to end:

1. The researcher opens the frontend URL in their browser. The frontend's JavaScript loads, the page renders.
2. The researcher types “find proteomics work on inflammatory skin conditions” and submits.
3. The frontend POSTs the query as JSON to `https://dash-api.<sub>.workers.dev/api/ask`.
4. The API Worker receives the request. It calls Cloudflare's Workers AI with the query text. Workers AI returns a 1024-element vector.
5. The API Worker sends a query to MongoDB Atlas asking for the projects whose stored vectors are closest to the query vector. Atlas returns the top matches with a similarity score.
6. The API Worker formats the results, builds a short prose answer (“I found 3 DASH projects related to ...”), and returns JSON to the frontend.
7. The frontend renders the answer as a chat message and the matches as cards. The researcher clicks a card to read more, which opens a side drawer with the full project details.

End to end, this takes roughly 300 to 400 milliseconds: about 200 ms for the embedding call, 50 ms for the database query, and the rest in network round trips.

## 3 Initial setup: the accounts and credentials

Setting up the mockup from scratch takes about an hour if you have credentials for the three external services. Most of the work is clicking through web dashboards. The actual code does not need to be modified at any point during initial setup.

You will need to set up, in order:

1. A Cloudflare account
2. A MongoDB Atlas account
3. A GitHub account, with a copy of the repository
4. A handful of secrets and tokens linking the three together

### 3.1 Cloudflare account

Sign up at [dash.cloudflare.com/sign-up](https://dash.cloudflare.com/sign-up). The free plan covers everything the mockup uses: Workers, Workers AI, and DNS.

Once you are signed in:

1. Note your **Account ID**, shown on the right sidebar of the Workers & Pages page. You will need this later.
2. Confirm you have access to **Workers AI** by visiting Workers & Pages → AI. New Cloudflare accounts have this enabled by default.
3. Pick a **workers.dev subdomain**. When you deploy your first Worker, Cloudflare prompts you to pick one. Choose something memorable; you cannot easily change it.

## 3.2 MongoDB Atlas account and cluster

Sign up at [mongodb.com/cloud/atlas/register](https://mongodb.com/cloud/atlas/register). Once in:

1. Create a new **Atlas Flex cluster**. Region: AWS Sydney (**ap-southeast-2**). Cluster name: anything; **dash** is fine. Atlas Flex costs about \$8 per month at our usage; the M0 free tier is not recommended for vector search.
2. Create a **database user** with read/write access to your **dash** database. Save the username and password.
3. Set **network access** to allow 0.0.0.0/0 (anywhere). This is mockup-acceptable because the connection is gated by username/password. Production should restrict to Cloudflare's egress IPs.
4. Note the cluster's **connection string** from the **Connect** dialog. It looks like:

```
mongodb+srv://<user>:<password>@dash.xxxxx.mongodb.net/?retryWrites=true&w=
majority
```

Replace **<user>** and **<password>** with the actual credentials. This whole string is what we call **ATLAS\_URI**.

## 3.3 Apply the Atlas indexes

The mockup expects three indexes on the **dash.projects** collection. Apply them via the Atlas dashboard → your cluster → Collections → Atlas Search and Indexes tabs:

1. **Vector search index** (**projects\_vector**): paste the JSON from **option\_b/atlas/vector-index.json** in the repo. This is the index that powers natural-language search.
2. **Compound indexes**: same idea, the JSON in **option\_b/atlas/compound-indexes.json**.
3. **Text search index** (**projects\_text**): the JSON in **option\_b/atlas/search-indexes.json**.

Each index takes a couple of minutes to build the first time. They self-maintain after that.

## 3.4 GitHub repository

If you are setting this up from scratch:

1. Clone the existing repo locally:

```
git clone https://github.com/ecool50/dash-repo-prototype.git
cd dash-repo-prototype
```

2. Either fork it to your own GitHub account, or transfer ownership to whichever organisation you want to host it under. We will refer to the new owner as **<org>** from here.

## 3.5 Generate the ingest secret

The CI/CD pipeline that pushes new projects into MongoDB calls a protected endpoint on the API Worker. The endpoint checks an **Authorization: Bearer ...** header. The token is something you generate yourself:

```
openssl rand -hex 32
```

This prints a 64-character random string. Save it somewhere safe (a password manager). The same value goes in two places, the Cloudflare Worker (as a secret) and GitHub Actions (as a secret).

### 3.6 Summary of what you should now have

Before continuing to deployment, you should have collected:

- A Cloudflare account, account ID noted
- A MongoDB Atlas cluster in Sydney, three indexes applied
- A MongoDB connection string (the `ATLAS_URI`)
- A copy of the repository under your control
- A 64-character random hex string (the ingest secret)

You will also need, but can get later:

- A **Cloudflare API token** with Workers Edit permission, for GitHub Actions to deploy on your behalf. We will create this in Section 6.
- The **deployed Worker URL**, for the frontend and for GitHub Actions to know where to send ingest requests. We will get this in Section 4.

## 4 Deploying the API Worker

The API Worker is the heart of the system: it serves the search endpoint that the frontend calls. Deploying it from a fresh laptop takes about ten minutes.

### 4.1 Install Wrangler

Wrangler is Cloudflare's command-line tool for managing Workers. The repo's `option_b/package.json` already pins a compatible version, so we install via npm:

```
cd option_b
npm install
```

This installs Wrangler and the MongoDB driver into `option_b/node_modules`.

### 4.2 Authenticate Wrangler with your Cloudflare account

```
cd option_b/api
npx wrangler login
```

This opens a browser, asks you to authorize Wrangler against your Cloudflare account, and stores credentials locally. You only have to do this once per machine.

### 4.3 Set the Worker secrets

The Worker needs two secret values: the MongoDB connection string and the ingest secret. Set them with Wrangler:

```
npx wrangler secret put ATLAS_URI
# Paste the full mongodb+srv://... string when prompted, press Enter.
```

```
npx wrangler secret put INGEST_SECRET
# Paste the 64-character random string from openssl, press Enter.
```

Both values are now stored encrypted on Cloudflare's side and made available to the Worker at runtime as `env.ATLAS_URI` and `env.INGEST_SECRET`. You cannot read them back; if you forget the value, you set a new one.

You can confirm both are present (without seeing the values) with:

```
npx wrangler secret list
```

The output should list `ATLAS_URI` and `INGEST_SECRET`.

## 4.4 Deploy the Worker

```
npx wrangler deploy
```

Wrangler bundles the code in `option_b/api/`, uploads it, and prints the deployed URL. On a fresh account this is the moment Cloudflare asks you to pick a `workers.dev` subdomain. After that, you see:

```
Uploaded dash-api (X.XX sec)
Published dash-api
https://dash-api.<your-subdomain>.workers.dev
```

That URL is now your live API. Save it; you will need it twice more during setup (once for the frontend's `.env.production`, once for a GitHub Actions secret).

## 4.5 Smoke test the deployed Worker

The repo includes a smoke test script that hits every endpoint and reports pass or fail:

```
cd ../../ # back to repo root
option_b/scripts/smoke.sh https://dash-api.<your-subdomain>.workers.dev
```

You should see PASS on six checks. If the database is empty (which it will be on a fresh Atlas cluster), the search endpoint will return zero results but still succeed; that's fine.

If anything fails here, see Section 9 on troubleshooting before continuing.

## 4.6 Load initial project data

If you want to load the ten illustrative projects right now, from the repo root:

```
ATLAS_URI='<your-mongodb-connection-string>' \
node option_b/scripts/ingest.mjs option_b/projects/A*.json
```

This ingests each JSON in turn. The script is idempotent: running it twice has the same effect as running it once. After this, the smoke test's search check should return actual matches.

# 5 Deploying the frontend

The frontend is a small React app built with Vite. It lives in `dash_frontend/` at the repo root. Deploying it to Cloudflare also takes about ten minutes.

## 5.1 Wire the frontend to the API

The frontend reads the API URL from an environment variable at build time. The file `dash_frontend/.env.production` sets this:

```
VITE_API_BASE_URL=https://dash-api.<your-subdomain>.workers.dev
```

If you are deploying on a different Cloudflare account than the one the repo was originally set up for, update this file with your own Worker URL before deploying.

## 5.2 Build and deploy the frontend Worker

The frontend is deployed as a Cloudflare Worker with an assets binding, which means the Worker serves the built static files. The configuration lives in `dash_frontend/wrangler.toml`. To deploy manually:

```
cd dash_frontend
npm install
npm run build
npx wrangler deploy
```

This builds the Vite bundle into `dash_frontend/dist/`, then uploads it as a Worker. The output prints the frontend URL:

```
Deployed dash-frontend triggers
https://dash-frontend.<your-subdomain>.workers.dev
```

Open that URL in a browser. You should see the DASH chat interface with a starter message and suggested queries.

## 5.3 Test a query end to end

Click one of the suggested queries, for example “find proteomics work on inflammatory skin conditions”. You should see a loading spinner, then a short agent message, then a grid of result cards. If you have loaded the ten illustrative projects, the top result is **A01: Proteomic profiling of a chronic inflammatory skin condition**.

Click “Open Details” on a card to see the side drawer with the full project description.

## 5.4 Setting up the deploy via Cloudflare’s dashboard

The above is the manual one-off deploy. To make subsequent deploys automatic (every push to GitHub redeploys the frontend), connect the frontend to GitHub via the Cloudflare dashboard:

1. Cloudflare → Workers & Pages → Create application → Connect to Git.
2. Pick your repository.
3. Project name: `dash-frontend`.
4. Root directory (sometimes called Path): `dash_frontend`.
5. Build command: `npm install && npm run build`.
6. Deploy command: `npx wrangler deploy`.
7. Save and deploy.

After this, every git push triggers a fresh build and deploy.



## 6 CI/CD: GitHub Actions for the API and the catalog

The repository has two GitHub Actions workflows that handle continuous deployment for the API Worker and continuous ingestion for new project metadata. Setting these up requires four GitHub secrets.

### 6.1 The two workflows in plain language

- `.github/workflows/deploy-worker.yml`: fires whenever code under `option_b/api/` changes on the main branch. It checks out the repo, installs dependencies, and runs `wrangler deploy`. Result: the API Worker is redeployed within a minute or two of every push that touches the API code.
- `.github/workflows/ingest-catalog.yml`: fires whenever any JSON file under `option_b/projects/` changes on main. It diffs the changed files, then POSTs each one to the deployed Worker's `/api/ingest` endpoint. The Worker validates, embeds, and writes to MongoDB. Result: any change to a project JSON is reflected in the search results within about a minute.

Both workflows need to authenticate against Cloudflare and the deployed Worker. They do this through four secrets stored in the GitHub repository settings.

### 6.2 Create a Cloudflare API token

The deploy workflow needs an API token to call Cloudflare on your behalf. In your Cloudflare account → My Profile → API Tokens → Create Token. Use the *Edit Cloudflare Workers* template, or build a custom policy with these scopes:

- Account → Workers Scripts: Read and Write
- Account → Workers AI: Read (optional but harmless)

When you click Create, the token value appears once. Copy it immediately into your notes; Cloudflare does not show it again.

### 6.3 Add the four secrets to GitHub

In your GitHub repository → Settings → Secrets and variables → Actions → New repository secret. Add the following four, exactly:

| Name                  | Value                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------------|
| CLOUDFLARE_API_TOKEN  | The token you just created.                                                               |
| CLOUDFLARE_ACCOUNT_ID | Your Cloudflare account ID from the dashboard sidebar.                                    |
| INGEST_URL            | The deployed API Worker URL, e.g. <code>https://dash-api.&lt;sub&gt;.workers.dev</code> . |
| INGEST_SECRET         | The same 64-character random hex string you set on the Worker.                            |

Names are case sensitive. Double-check before saving each one.

### 6.4 Verify the pipeline

The easiest verification is to push something. From the repo root:

```
# Touch any JSON's updated_at to trigger the ingest workflow:
node -e "const fs=require('fs');const f='option_b/projects/A03.json';const d=
  JSON.parse(fs.readFileSync(f,'utf8'));d.updated_at=new Date().toISOString();
  fs.writeFileSync(f,JSON.stringify(d,null,2)+'\n')"
```

```
git add option_b/projects/A03.json
git commit -m "Touch A03 to verify ingest workflow"
git push origin main
```

In the GitHub Actions tab you should see a run of **Ingest catalog changes**. Click into it and expand the POST step; you should see a single **ok (200)** line for A03.

If the workflow fails, the log will show one of three things:

- The secrets-validation step flagged an empty or malformed secret. Fix it in repo Settings.
- The curl call failed with a non-2xx status. The body is printed; check whether it's a 401 (auth, secret mismatch) or a 500 (Worker exception, look at the Worker logs).
- The script could not resolve the host. Likely **INGEST\_URL** is missing the **https://** prefix or has stray whitespace.

## 7 Day-to-day operations

Once the system is set up, the routine activities are: adding new projects, occasionally editing the API code, and editing the frontend.

### 7.1 Adding or updating a project

The catalogue is a set of JSON files under **option\_b/projects/**. To add a new project:

1. Copy **JSON\_template\_V1.json** from the repo root (or any existing project file) as a starting point.
2. Fill in the required fields. At a minimum: **ref\_number**, **title**, **status**, the project details (research area, data modality, sample info, keywords), the methods (primary methods, tools, programming languages), and the executive summary. The schema in **option\_b/schema\_v1.json** is the authority on what's required.
3. Save the file as **option\_b/projects/<ref\_number>.json**.
4. Validate against the schema before committing:

```
node option_b/scripts/validate.mjs option_b/projects/<ref_number>.json
```

5. Commit and push.

The ingest workflow fires automatically on push. It validates the file again, generates the embedding source text, calls Workers AI for a vector, and writes the document into MongoDB. The project becomes searchable within about a minute of the merge.

To update an existing project, simply edit its JSON file and push. The workflow's smart re-embedding logic regenerates the vector only if the embedding source text (title, executive summary, methods, modality, disease, keywords, tools) has changed. Updates to other fields like **updated\_at** or **outputs.report\_url** skip the embedding call entirely.

### 7.2 Editing the API code

The API code lives in **option\_b/api/**. The four files that contain runtime logic are:

- **worker.js**: HTTP entry point, route dispatching.

- **search.js**: vector search pipeline.
- **ask.js**: frontend-facing wrapper that synthesises an answer.
- **ingest.js**: schema validation, embedding generation, upserts.

To make a change locally and ship it:

1. Edit the file.
2. If you want to test locally, run `npx wrangler dev` from `option_b/api/`. This starts a local server that proxies to the live Cloudflare AI binding but talks to the live Atlas cluster.
3. Commit and push.
4. The deploy workflow rebuilds and redeploys within a couple of minutes.

To roll back: `npx wrangler rollback` from `option_b/api/` reverts to the previous deployed version.

### 7.3 Editing the frontend

The frontend code lives in `dash_frontend/src/app.jsx` and the styles in `src/styles.css`. The build process is standard Vite.

To make a change locally:

```
cd dash_frontend
npm install    # only needed first time
npm run dev    # starts a local dev server on http://localhost:5234
```

The dev server talks to the live API by default (because `VITE_API_BASE_URL` resolves to the production URL). You can override it by creating `.env.local` with a different value.

To ship the change, commit and push. If Pages is connected to GitHub (Section 5.4), it redeploys automatically. Otherwise: `npm run build && npx wrangler deploy` from `dash_frontend/`.

### 7.4 Running the smoke test

After any deployment, run the smoke test to confirm nothing regressed:

```
option_b/scripts/smoke.sh https://dash-api.<your-subdomain>.workers.dev
```

It hits all five endpoints in sequence and reports pass or fail. With the optional `INGEST_SECRET` environment variable set, it also tests the authenticated ingest endpoint.

### 7.5 Viewing logs

- **Live Worker logs**: `npx wrangler tail` from `option_b/api/`. Streams request logs in real time.
- **Deploy logs**: GitHub repo → Actions tab. Each workflow run has full logs.
- **Atlas logs**: MongoDB Atlas dashboard → your cluster → Metrics. Slow queries, connection counts, storage usage.

## 8 Migrating to DASH-owned infrastructure

When the mockup needs to move off your personal accounts onto DASH-owned infrastructure, the work is mostly clicking through dashboards. The code does not need to change.

### 8.1 What moves and what does not

| Component              | Migration action                                                                                      |
|------------------------|-------------------------------------------------------------------------------------------------------|
| GitHub repository      | Transfer ownership to the DASH org in repo Settings. Commits, branches, workflows are all preserved.  |
| API Worker             | Redeploy from the new Cloudflare account with <code>wrangler deploy</code> . Code is unchanged.       |
| Frontend               | Same as above. Update <code>.env.production</code> if the API URL changes.                            |
| Worker secrets         | Run <code>wrangler secret put</code> for each secret again on the new account.                        |
| MongoDB Atlas cluster  | Either transfer billing to DASH on the existing cluster, or migrate data to a new DASH-owned cluster. |
| GitHub Actions secrets | Set the four secrets again in the new repo's Settings.                                                |
| Custom domain          | Configure DNS on the DASH-owned Cloudflare account.                                                   |

### 8.2 The two Atlas options

**Option A: transfer billing.** Atlas lets you move billing for an existing project to a different account. The cluster stays put, the connection string stays the same, no data movement. The DASH account becomes responsible for paying for the cluster. The Worker does not need to be redeployed.

**Option B: migrate data to a new cluster.** The DASH side creates a fresh Atlas project and cluster. You use `mongodump` to export the data from the old cluster and `mongorestore` to import it into the new one. You update `ATLAS_URI` on the Worker and redeploy. The data set is small (under 100 MB at 1,000 projects) so the migration itself takes about ten minutes once the credentials are sorted.

Option A is simpler when both accounts can coexist on the same Atlas project. Option B is cleaner when DASH wants its own Atlas project from the start.

### 8.3 The full migration sequence

1. **DASH side:** create the Cloudflare account (or get team access to the institutional one); create the Atlas project (or transfer billing).
2. **You:** transfer the GitHub repository to the DASH org via GitHub repo Settings. All workflows, branches, and history come with it.
3. **You, signed in to the new Cloudflare account:** from `option_b/api/`, run `npx wrangler secret put ATLAS_URI` (with the new or transferred connection string) and `npx wrangler secret put INGEST_SECRET`. Then `npx wrangler deploy`. Note the new URL.
4. **You, signed in to the new Cloudflare account:** from `dash_frontend/`, update `.env.production` if the API URL changed, then `npm install && npm run build && npx wrangler deploy`.
5. **DASH GitHub admin:** in the new repo's Settings → Secrets and variables → Actions, add the four secrets again with new values where they have changed.
6. **You:** run the smoke test against the new Worker URL.

7. **You:** make one trivial JSON edit and push to confirm the ingest workflow fires on the new repo.

Total time end to end: roughly half a day, mostly waiting for builds.

## 8.4 Why this is straightforward

The architecture has these portability properties baked in:

- No hardcoded account IDs anywhere in code. The Cloudflare account is set at deploy time via the API token, not in any committed file.
- Workers are stateless. All persistent state lives in Atlas. Redeploying from a fresh Cloudflare account creates an identical service.
- All sensitive configuration is in encrypted secret stores. Migration is “set them again,” not “change code.”
- GitHub Actions workflow files are repo-local. They work wherever the repo lives.
- The only place a URL is hardcoded is `dash_frontend/.env.production`. That’s one file, one line.

## 9 Troubleshooting

When something stops working, the failure mode is almost always one of a small number of common causes. This section covers the most likely.

### 9.1 The frontend loads but no results come back

Look at the browser’s developer tools → Network tab. Click your search request. The likely failures:

- **CORS error:** the Worker’s `ALLOWED_ORIGIN` environment variable is too restrictive. Default is `*`; if you tightened it, make sure the frontend’s origin matches.
- **500 with “Invalid scheme”:** the `ATLAS_URI` secret on the Worker is empty or malformed. Re-run `wrangler secret put ATLAS_URI` from `option_b/api/`.
- **500 with “ReadPreference is not a constructor”:** the deploy was run with wrangler 3.x. The MongoDB driver requires wrangler 4.x; make sure Node 22+ is being used.
- **200 but empty results:** the database has no documents (run the ingest script) or no projects match the query.

### 9.2 The deploy workflow fails

Common causes:

- **Wrong Cloudflare API token scope:** the token needs Workers Edit. Create a new one with the correct scope and update the GitHub secret.
- **Wrong account ID:** `CLOUDFLARE_ACCOUNT_ID` secret has a typo or extra characters. Copy it again from the Cloudflare dashboard sidebar.
- **Node version mismatch:** wrangler 4.x requires Node 22+. The workflow uses Node 22 explicitly; if you customised it to a lower version, raise it.

- **Mongo bundling error:** wrangler 3.x cannot bundle the MongoDB driver correctly. Make sure the workflow uses Node 22 so wrangler 4 is picked up.

### 9.3 The ingest workflow fails

Common causes, ordered by frequency:

- **INGEST\_URL malformed or empty:** the secret-validation step flags this with a clear error. Fix the value in repo Settings.
- **401 Unauthorized:** the INGEST\_SECRET on GitHub and the Worker do not match. Set them to the same value in both places.
- **500 with schema validation error:** the JSON does not pass schema validation. The error message includes the field name. Fix the JSON and push again.

### 9.4 Atlas connection refused or timeout

- **Network Access not configured:** Atlas → Security → Network Access. The Worker's egress IPs need to be allowed. For mockup, use 0.0.0.0/0.
- **Database user credentials wrong:** Atlas → Security → Database Access. Confirm the username and password in ATLAS\_URI match an actual user.

## 10 Reference

### 10.1 File and directory layout

```
repo_project/
|-- option_b/                # the Phase 1 mockup
|   |-- api/                 # Cloudflare Worker source
|   |   |-- worker.js        # HTTP entry point and routing
|   |   |-- search.js        # vector search pipeline
|   |   |-- ask.js           # frontend-facing wrapper
|   |   |-- ingest.js        # schema validation, embed, upsert
|   |   |-- mongo.js         # MongoDB client wrapper
|   |   |-- wrangler.toml    # Worker config
|   |-- projects/            # the catalogue: one JSON per project
|   |-- atlas/               # Atlas index specifications (JSON)
|   |-- scripts/             # smoke test, ingest helper, validators
|   |-- docs/                # documents like this one
|   |-- demo.md              # demo script for supervisor presentations
|   |-- schema_v1.json        # canonical schema for project JSONs
|   |-- package.json         # Node deps (mongodb, wrangler)
|   |-- README.md
|-- dash_frontend/           # React + Vite frontend
|   |-- src/                 # app.jsx, styles.css
|   |-- index.html
|   |-- vite.config.js
|   |-- wrangler.toml        # Worker assets config
|   |-- package.json
|   |-- .env.production      # VITE_API_BASE_URL
|-- option_a/                # earlier prototype, archival
|-- .github/workflows/       # GitHub Actions
|   |-- deploy-worker.yml    # redeploys API on push
|   |-- ingest-catalog.yml   # POSTs changed JSONs to /api/ingest
```

## 10.2 HTTP endpoints exposed by the API Worker

| Method | Path              | Purpose                                                      |
|--------|-------------------|--------------------------------------------------------------|
| POST   | /api/query-embed  | Convert text to a 1024-element vector.                       |
| POST   | /api/search       | Vector search; returns <code>{results}</code> .              |
| POST   | /api/ask          | Frontend wrapper; returns <code>{answer, matches}</code> .   |
| GET    | /api/projects/:id | Fetch a single project document by <code>ref_number</code> . |
| POST   | /api/ingest       | Authenticated; validate, embed, upsert a project JSON.       |

## 10.3 Secrets and where they are set

| Name                  | Stored in                                                    | Used by                                                                                        |
|-----------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| ATLAS_URI             | Cloudflare Worker secret store                               | Worker, at every database call.                                                                |
| INGEST_SECRET         | Cloudflare Worker secret store + GitHub Actions secret store | Worker ( <code>env.INGEST_SECRET</code> ) and the ingest workflow's bearer header; must match. |
| CLOUDFLARE_API_TOKEN  | GitHub Actions secret store                                  | The deploy workflow's <code>wrangler-action</code>                                             |
| CLOUDFLARE_ACCOUNT_ID | GitHub Actions secret store                                  | The deploy workflow's <code>wrangler-action</code>                                             |
| INGEST_URL            | GitHub Actions secret store                                  | The ingest workflow's curl call.                                                               |

## 10.4 What goes into the embedding source text

The Worker builds the text it sends to Workers AI by concatenating these fields, in order, with the separator `|`:

1. `title`
2. `findings.executive_summary`
3. `analytical_methods.primary_methods` (joined with spaces)
4. `project_details.data_modality` (joined with spaces)
5. `project_details.disease` (joined with spaces)
5. `project_details.keywords` (joined with spaces)
6. `analytical_methods.tools_packages` (joined with spaces)

A change to any other field, like `n_samples` or `outputs.github_repo`, does not trigger re-embedding. This keeps the vector stable when fields irrelevant to search are updated.

## 10.5 Schema enums

For quick reference:

- `status`: `active`, `complete`, `on_hold`, `archived`
- `access_tier`: `private`, `dash_internal`, `dash_extended`, `cpc_engaged`, `cpc_all`, `usyd`, `public`
- `access_preset`: `draft`, `shared`, `published`
- `report_layout`: `single`, `multiple`

---

This document describes the system as of June 2026. If the architecture changes (for example, the embedding model is swapped, or RBAC is activated), update the relevant section here so the next person to pick this up does not have to reverse-engineer from the code.